
Algorithms I

BA4 • 2025–2026

Contents

1	Algorithms Fundamentals	4
1.1	Definition of an Algorithm	4
1.2	Asymptotic Complexity	4
1.3	Sorting Algorithms	4
1.3.1	Insertion Sort	4
1.3.2	Merge Sort	5
1.4	Divide and Conquer	5
1.4.1	Maximum Subarray	5
1.4.2	Matrix Multiplication	5
1.4.3	Karatsuba Multiplication	6
1.5	Heaps	6
1.5.1	Heapify	6
1.5.2	Build-Heap	6
1.5.3	Heapsort	7
1.5.4	Priority Queue	7
1.6	Binary Search Trees	7
1.7	Data Structures Comparison	8
2	Graph Algorithms — Max Flow	9
2.1	Flow Networks	9
2.2	Ford-Fulkerson Method	9
2.3	Applications	9
2.3.1	Bipartite matching via max flow	10
2.3.2	Edge-disjoint paths via max flow	10
3	Quizzes	11
3.1	Quiz #7	11
3.1.1	Bipartite matching via max flow	11
3.1.2	Edge disjoint paths via max flow	11
3.2	Quiz #9	12
3.2.1	Bellman-Ford	12
3.2.2	Dijkstra	13

These notes cover the core algorithmic techniques of the BA4 Algorithms I course: asymptotic analysis, sorting, divide-and-conquer, heaps, binary search trees, and graph algorithms (max flow, matchings).

CHAPTER 1

Algorithms Fundamentals

1.1 Definition of an Algorithm

Definition

An **algorithm** is a well-defined computational procedure that takes some value (or set of values) as *input* and produces some value (or set of values) as *output* in a finite number of steps.

We analyse algorithms along two axes:

- **Correctness** — does it always produce the right answer?
- **Efficiency** — how many resources (time, space) does it consume?

1.2 Asymptotic Complexity

Asymptotic notations

Let $f, g : \mathbb{N} \rightarrow \mathbb{R}^+$.

- $f \in O(g)$ (**upper bound**): $\exists c, n_0 > 0$ s.t. $f(n) \leq c g(n)$ for all $n \geq n_0$
- $f \in \Omega(g)$ (**lower bound**): $\exists c, n_0 > 0$ s.t. $f(n) \geq c g(n)$ for all $n \geq n_0$
- $f \in \Theta(g)$ (**tight bound**): $f \in O(g)$ and $f \in \Omega(g)$
- $f \in o(g)$ (**strict upper**): $\lim_{n \rightarrow \infty} f(n)/g(n) = 0$
- $f \in \omega(g)$ (**strict lower**): $\lim_{n \rightarrow \infty} f(n)/g(n) = \infty$

Common growth rates (slowest to fastest)

$$O(1) \subset O(\log n) \subset O(n) \subset O(n \log n) \subset O(n^2) \subset O(n^3) \subset O(2^n) \subset O(n!)$$

Remark

For the **Master Theorem**: given $T(n) = aT(n/b) + f(n)$ with $a \geq 1, b > 1$:

- if $f(n) = O(n^{\log_b a - \varepsilon})$ then $T(n) = \Theta(n^{\log_b a})$
- if $f(n) = \Theta(n^{\log_b a})$ then $T(n) = \Theta(n^{\log_b a} \log n)$
- if $f(n) = \Omega(n^{\log_b a + \varepsilon})$ then $T(n) = \Theta(f(n))$

1.3 Sorting Algorithms

1.3.1 Insertion Sort

Complexity

Time: $\Theta(n^2)$ worst/average, $\Theta(n)$ best (already sorted). Space: $O(1)$ in-place.

```
1 def insertion_sort(A):  
2     for j in range(1, len(A)):
```

```

3     key = A[j]
4     i = j - 1
5     while i >= 0 and A[i] > key:
6         A[i + 1] = A[i]
7         i -= 1
8     A[i + 1] = key

```

Loop invariant: at the start of each iteration, $A[0..j-1]$ is sorted.

1.3.2 Merge Sort

Complexity

Time: $\Theta(n \log n)$ in all cases. Space: $O(n)$ auxiliary.

```

1 def merge_sort(A, p, r):
2     if p < r:
3         q = (p + r) // 2
4         merge_sort(A, p, q)
5         merge_sort(A, q + 1, r)
6         merge(A, p, q, r) # merge two sorted halves

```

Recurrence: $T(n) = 2T(n/2) + \Theta(n) \Rightarrow T(n) = \Theta(n \log n)$ by Master Theorem (case 2).

1.4 Divide and Conquer

Paradigm

Divide the problem into subproblems, **conquer** each recursively, then **combine** the results.

1.4.1 Maximum Subarray

Find indices $i \leq j$ maximising $\sum_{k=i}^j A[k]$.

Complexity

$T(n) = 2T(n/2) + \Theta(n) \Rightarrow \Theta(n \log n)$. (Kadane's linear algorithm also exists: $\Theta(n)$.)

```

1 FIND-MAXIMUM-SUBARRAY(A, low, high):
2     if high == low: return (low, high, A[low])
3     mid = (low + high) / 2
4     left = FIND-MAXIMUM-SUBARRAY(A, low, mid)
5     right = FIND-MAXIMUM-SUBARRAY(A, mid+1, high)
6     cross = FIND-MAX-CROSSING(A, low, mid, high)
7     return the one with maximum sum

```

1.4.2 Matrix Multiplication

Naive algorithm

Standard $O(n^3)$ triple loop, or the recursive divide-and-conquer that splits each $n \times n$ matrix into four $n/2 \times n/2$ blocks: $T(n) = 8T(n/2) + \Theta(n^2)$, giving the same $\Theta(n^3)$ by Master Theorem (case 1).

*Strassen's algorithm***Complexity**

$$T(n) = 7T(n/2) + \Theta(n^2) \Rightarrow \Theta(n^{\log_2 7}) \approx \Theta(n^{2.807}).$$

Key idea: compute only 7 products of submatrices (instead of 8) using the following auxiliary matrices $M_1, \dots, M_7$, each a linear combination of blocks of A and B :

$$M_1 = (A_{11} + A_{22})(B_{11} + B_{22}), \quad M_2 = (A_{21} + A_{22})B_{11}, \quad \dots$$

Then reconstruct $C_{11}, C_{12}, C_{21}, C_{22}$ from M_1-M_7 .

1.4.3 Karatsuba Multiplication

Multiply two n -digit integers faster than the naive $O(n^2)$ schoolbook method.

Split: $x = x_H b^{n/2} + x_L$, $y = y_H b^{n/2} + y_L$.

Compute only 3 multiplications:

$$p_1 = x_H \cdot y_H, \quad p_2 = x_L \cdot y_L, \quad p_3 = (x_H + x_L)(y_H + y_L)$$

then $x \cdot y = p_1 b^n + (p_3 - p_1 - p_2) b^{n/2} + p_2$.

Complexity

$$T(n) = 3T(n/2) + \Theta(n) \Rightarrow \Theta(n^{\log_2 3}) \approx \Theta(n^{1.585}).$$

1.5 Heaps**Max-heap property**

A **max-heap** is a nearly complete binary tree where every node satisfies $A[\text{parent}(i)] \geq A[i]$. Stored as an array: $\text{LEFT}(i) = 2i$, $\text{RIGHT}(i) = 2i + 1$, $\text{PARENT}(i) = \lfloor i/2 \rfloor$.

1.5.1 Heapify**Complexity**

$O(\log n)$ — tree height.

```

1 MAX-HEAPIFY(A, i, n):
2   l = 2i;  r = 2i + 1;  largest = i
3   if l <= n and A[l] > A[largest]: largest = l
4   if r <= n and A[r] > A[largest]: largest = r
5   if largest != i:
6       swap A[i] and A[largest]
7       MAX-HEAPIFY(A, largest, n)
```

1.5.2 Build-Heap

Call MAX-HEAPIFY on all internal nodes bottom-up. Though each call is $O(\log n)$, a tight analysis gives $O(n)$ total.

1.5.3 Heapsort

Complexity

Time: $\Theta(n \log n)$ in all cases. Space: $O(1)$ in-place.

```

1 HEAPSORT(A, n):
2   BUILD-MAX-HEAP(A, n)
3   for i = n downto 2:
4       swap A[1] and A[i]
5       MAX-HEAPIFY(A, 1, i - 1)
```

1.5.4 Priority Queue

Property

Maximum	$O(1)$
Extract-Max	$O(\log n)$
Insert	$O(\log n)$
Increase-Key	$O(\log n)$

1.6 Binary Search Trees

BST property

For every node x : all keys in the left subtree $\leq x.key \leq$ all keys in the right subtree.

All basic operations run in $O(h)$ where h is the height. For a *random* BST, $\mathbb{E}[h] = O(\log n)$; in the worst case $h = n - 1$.

```

1 TREE-SEARCH(x, k):
2   if x == NIL or k == x.key: return x
3   if k < x.key: return TREE-SEARCH(x.left, k)
4   else:         return TREE-SEARCH(x.right, k)
```

Operations — all $O(h)$

Search, Minimum, Maximum, Successor, Predecessor, Insert, Delete.

Warning

A BST is only efficient when *balanced*. For guaranteed $O(\log n)$, use a self-balancing variant such as a **Red-Black tree** (height $\leq 2 \log_2(n + 1)$).

1.7 Data Structures Comparison

Structure	Search	Insert	Delete	Notes
Unsorted array	$O(n)$	$O(1)$	$O(n)$	simple
Sorted array	$O(\log n)$	$O(n)$	$O(n)$	binary search
Linked list	$O(n)$	$O(1)$	$O(1)^*$	*with pointer
BST	$O(h)$	$O(h)$	$O(h)$	$h = O(n)$ worst
Red-Black tree	$O(\log n)$	$O(\log n)$	$O(\log n)$	balanced
Heap	$O(n)$	$O(\log n)$	$O(\log n)$	max/min in $O(1)$
Hash table	$O(1)^*$	$O(1)^*$	$O(1)^*$	*expected

CHAPTER 2

Graph Algorithms — Max Flow

2.1 Flow Networks

Flow network

A directed graph $G = (V, E)$ with:

- a **source** s and a **sink** t ,
- a **capacity function** $c : V \times V \rightarrow \mathbb{R}_{\geq 0}$ (0 for non-edges),
- no antiparallel edges (replace (u, v) and (v, u) by splitting with an intermediate node).

A **flow** f must satisfy capacity ($0 \leq f(u, v) \leq c(u, v)$) and conservation ($\sum_v f(v, u) = \sum_v f(u, v)$ for all $u \neq s, t$).

The **value** of a flow is $|f| = \sum_v f(s, v) - \sum_v f(v, s)$.

2.2 Ford-Fulkerson Method

Residual network

Given flow f , the residual network G_f has residual capacity $c_f(u, v) = c(u, v) - f(u, v)$ for forward edges and $c_f(v, u) = f(u, v)$ for back edges. An **augmenting path** is any s - t path in G_f .

```
1 FORD-FULKERSON(G, s, t):  
2   initialise f = 0 on all edges  
3   while exists augmenting path p in G_f:  
4       cf(p) = min capacity along p  
5       augment f along p by cf(p)  
6   return f
```

Max-flow min-cut theorem

The following are equivalent for a flow f :

1. f is a maximum flow.
2. There is no augmenting path in G_f .
3. $|f|$ equals the capacity of some minimum cut (S, T) .

2.3 Applications

2.3.1 Bipartite matching via max flow

Algorithm

Goal: Maximum matching in a bipartite graph $G = (L \cup R, E)$.

Reduction:

- Add source s with edge $s \rightarrow u$ (capacity 1) for each $u \in L$.
- Keep all edges $u \rightarrow v$ for $(u, v) \in E$ (capacity 1).
- Add sink t with edge $v \rightarrow t$ (capacity 1) for each $v \in R$.

Run max-flow. Each unit of flow corresponds to one matched pair. The maximum matching has size $|f^*|$.

2.3.2 Edge-disjoint paths via max flow

Algorithm

Goal: Maximum number of s - t paths sharing no edge.

Reduction: Keep the original graph, set every edge capacity to 1. Run max-flow from s to t . The value $|f^*|$ equals the maximum number of edge-disjoint paths. By the max-flow min-cut theorem, it also equals the minimum number of edges whose removal disconnects s from t .

CHAPTER 3

Quizzes

3.1 Quiz #7

3.1.1 Bipartite matching via max flow

Bipartite matching via max flow — student notes

Cet algorithme est une application de max flow : même code en substance, mais le contexte et les règles du graphe changent.

Qu'est-ce que le bipartite matching ? On a deux sets d'éléments et le but est de faire le maximum de paires possibles, sachant que chaque élément ne peut être apparié qu'une seule fois avec un élément de l'autre set, et uniquement avec certains.

Modélisation du graphe :

src $\rightarrow$ tous les éléments du premier set (capacité 1)
éléments set 1 $\rightarrow$ éléments set 2 compatibles (capacité 1)
éléments set 2 $\rightarrow$ end (capacité 1)

La capacité 1 sur chaque branche garantit qu'elle n'est traversée qu'une fois, évitant ainsi les doublons ou les mauvais appariements.

3.1.2 Edge disjoint paths via max flow

Edge disjoint paths via max flow — student notes

On utilise max flow pour trouver le maximum de chemins de s à t sans intersection.

Qu'est-ce que edge disjoint paths ? On cherche le maximum de chemins de A à B qui ne partagent aucune arête.

Modélisation du graphe :

chaque chemin = arêtes du graphe
capacité de chaque arête = 1

La capacité 1 force chaque arête à n'être empruntée qu'une fois. Le max flow donne alors directement le nombre maximum de chemins arête-disjoints.

3.2 Quiz #9

3.2.1 Bellman-Ford

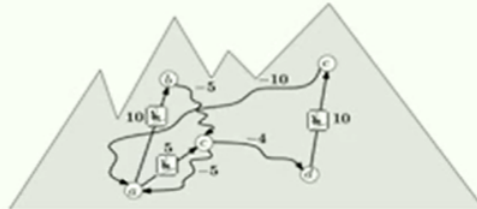
1. **Pourquoi $|V| - 1$ itérations sont suffisantes :** Le nombre $|V| - 1$ correspond au nombre maximal d'arêtes dans un chemin simple. Après $|V| - 1$ itérations, l'algorithme a nécessairement trouvé tous les plus courts chemins ne contenant aucun cycle.
2. **Détection des cycles négatifs :** Si le coût d'un chemin peut encore être réduit lors de la $|V|$ -ième itération, cela implique que le plus court chemin contient plus de $|V| - 1$ arêtes. Ceci n'est possible que si le chemin contient un cycle, et pour que le coût diminue, ce cycle doit être négatif.

Problème d'examen vu en cours

Problem solving (exam)

Problem solving (20 pts * exam question ~~last~~ ^{previous} year)

The famous alpine ski racer Lindsey Vonn has sent her assistant to measure the altitude differences of the ski lifts and slopes of a famous ski resort in Switzerland. The assistant returns after several days of hard work with a map of all ski lifts and all slopes together with the altitude difference between the start station and the end station of each lift and the altitude difference between the start station and end station of each slope. A slope starts from the end station of a lift and ends at the start station of a potentially different lift (see the figure below). Lindsey wants to verify the map of her assistant by performing the following sanity check: starting from any point A , no matter how we ski (using lifts and slopes), the altitude change when we return to A should be 0 (i.e., neither strictly negative nor strictly positive). Design an algorithm to perform this sanity check and analyze its running time in terms of the number of slopes and ski lifts ($|E|$) and the number of start and end stations ($|V|$). Your algorithm should run in polynomial time (in $|V|$ and $|E|$).



Idée. La carte est cohérente si et seulement si tout cycle C du graphe vérifie $W(C) = 0$. On décompose cette condition en deux :

$$\forall C : W(C) = 0 \iff (\forall C : W(C) \geq 0) \text{ ET } (\forall C : W(C) \leq 0)$$

Chacune se vérifie avec Bellman-Ford en détectant l'absence de cycle négatif : une fois avec les poids w , une fois avec $-w$.

Pourquoi $|V| - 1$ itérations suffisent. Un plus court chemin sans cycle a au plus $|V| - 1$ arêtes. Après $|V| - 1$ itérations de relaxation, toutes les distances correctes sont calculées.

- **Pas de cycle négatif :** tout est stabilisé après $|V| - 1$ itérations ; une $|V|$ -ème ne relaxera plus rien.

- **Cycle négatif présent** : on peut indéfiniment réduire une distance en repassant par le cycle, donc la $|V|$ -ème itération relaxe encore au moins une arête.

En exécutant Bellman-Ford deux fois (une fois avec w , une fois avec $-w$), la solution complète reste $O(|V| \cdot |E|)$.

3.2.2 Dijkstra

Ce qu'il faut retenir

L'idée centrale. À chaque étape on finalise le sommet non-visité dont la distance estimée est minimale. Cette distance est déjà optimale — car tous les poids sont ≥ 0 , aucun chemin futur ne pourra la réduire. C'est exactement ce qui distingue Dijkstra de Bellman-Ford : pas de deuxième chance, pas de retour en arrière.

Ce qui se passe à chaque itération.

1. On extrait $u = \arg \min_{v \in Q} v.d$ de la file.
2. On l'ajoute à S — sa distance est maintenant définitive.
3. Pour chaque voisin v de u : si $u.d + w(u, v) < v.d$, on met à jour $v.d$ et $v.\pi$.

Pourquoi ça ne marche pas avec des poids négatifs. Si une arête négative (x, y) existe, un sommet x déjà dans S pourrait offrir un chemin plus court vers y mais y est peut-être déjà finalisé.